Supplementary Material

Failure-Aware Container Orchestration under Node and Network Partitions in Resource-Constrained Healthcare Cloud Environments

Kawuma Daniel and Fred Wabendo

This supplement holds the four comparative matrices of the literature review (Section S1, Tables S1–S5) and the run-level OpenMRS tables summarised in the main paper (Section S2, Tables S6–S12). Reference numbers match the main paper. Every run-level value is also in the run directories and CSV files of the repository release.

S1 Comparative Systems Analysis

Primary search string. Run in IEEE Xplore, Google Scholar and the ACM Digital Library on 22–23 August 2026; all 20 queries are in the search log.

> ("container orchestration" OR "container orchestrator" OR Kubernetes OR K8s OR "container scheduling" OR "containerized workload*" OR "cloud orchestration" OR "edge orchestration") AND ("node failure" OR "host failure" OR "server failure" OR "node crash" OR "network partition" OR "network failure" OR "connectivity failure" OR "link failure" OR "packet loss" OR "resource exhaustion" OR "resource saturation" OR "resource contention" OR "resource pressure" OR "memory exhaustion" OR "memory pressure" OR "out of memory" OR "CPU saturation" OR "CPU contention" OR "failure detection" OR "failure recovery" OR "fault tolerance" OR "failure-aware" OR resilien*) AND ("resource constrained" OR "resource-constrained" OR "constrained infrastructure" OR "limited resources" OR "distributed cloud" OR "multi-site" OR "geo-distributed" OR "geographically distributed" OR "intermittent connectivity" OR "disconnected operation" OR "offline operation" OR "wide area network" OR WAN)

This section presents four comparative matrices covering all 24 included papers. In Matrices 1–3 the rows below the rule are the three papers whose decision component is learned [29–31]; they are coded on the same dimensions as the others, and Section III of the main paper explains how they inform the design. Cross-paper trade-off discussion follows each matrix.

Matrix 1 – Multi-Dimensional Taxonomy Mapping.

The comparative matrices use the following terms after introducing them here in full: wide-area network (WAN), Domain Name System (DNS), service-level agreement (SLA), machine learning (ML), convolutional neural network (CNN), long short-term memory (LSTM), graphics processing unit (GPU), multi-instance GPU (MIG), genetic algorithm (GA), peer-to-peer (P2P), distributed hash table (DHT), mean time to detect (MTTD), mean time to recover (MTTR), 95th-percentile latency (P95), 99th-percentile latency (P99), exponentially weighted moving average (EWMA), support vector classifier (SVC), Google Kubernetes Engine (GKE), recovery point objective (RPO), online transaction processing (OLTP), trusted execution environment (TEE), fifth-generation mobile networking (5G), Internet of Things (IoT), and artificial intelligence (AI). The notation N/A means not applicable.

Read across substrate and implementation-layer columns, the corpus concentrates heavily on OS-level containers with user-space middleware implementation, the same category the implemented mechanism in Section VII of the main paper occupies; only three papers, [8], [26] and [25], reach into control-plane or kernel-level modification, the latter two citing minimal intrusive change as an explicit design goal, consistent with this project's own scope boundary.

Matrix 2 – Empirical Evaluation, Workload, and Data Trace.

Matrix S2 additionally uses Google Cloud Platform (GCP), Amazon Web Services (AWS), round-trip time (RTT), User Datagram Protocol (UDP), Yahoo! Cloud Serving Benchmark (YCSB), input/output (I/O), Advanced Micro Devices (AMD), and Secure Encrypted Virtualization–Secure Nested Paging (SEV-SNP). Product and benchmark names are retained as published by their respective authors.

The fault-parameters column is the clearest evidence for the gap formalised in Section VII of the main paper: of the 24 coded papers, only [7], [14], [23], [25], and [27] inject anything resembling a genuine network partition, and of these, only [23] and [27] separately compare partition against crash on identical infrastructure. None of these partition-injection studies evaluates a general-purpose Kubernetes decision stage that combines post-suspicion

Table S1: Matrix 1: Taxonomy mapping across the 24 included papers. Rows below the rule have a learned decision component.

Paper	Substrate	Objective	Decision mechanism	Impl. layer	Scaling behaviour	Evaluation method	Domain
[4]	OS container (vanilla)	Pod/API/DNS latency	N/A (stock K8s)	Kernel/interface	N/A	Bare-metal, WAN emulation	Edge, general
[5]	OS container + Kafka	Recovery speed, SLA	Hybrid: quorum + ML ensemble	Streaming/inference/control	Adaptive hybrid	Public-cloud clusters (GCP, AWS)	Manufacturing
[6]	OS container (K8s/K3s)	SLA risk, MTTR	Predictive loop (LSTM + policy)	User-space middleware	Predictive	Simulation	Telecom/NFV
[7]	OS container (K8s)	Response time, success rate	N/A (stock K8s recovery)	Control + data plane	Failure-triggered	Emulation + bare-metal	General cloud/edge
[8]	OS container (K8s/Swarm)	Orchestration/failover time	Static, fixed-priority	Control-plane core	Failure-triggered	Bare-metal, two testbeds	Real-time/5G
[9]	OS container (Docker)	Task placement latency	Decentralized consensus	User-space middleware	N/A	Bare-metal (Pi cluster)	Edge microservices
[10]	OS container (custom orch.)	Overload time, energy	Hybrid: CNN+LSTM + threshold	Distributed control plane	Predictive	Heterogeneous edge testbed	Healthcare/hospital
[11]	Container + GPU (MIG)	Job completion time	Hybrid ML (LSTM + GA)	Scheduler extender	Predictive+placement	Real testbed + trace replay	Hyperscale ML training
[12]	Custom P2P/DHT agent	Scheduling/recovery latency	Decentralized lease + DHT	User-space, custom	N/A	Simulation	General distributed
[13]	OS container (K8s)	MTTD, MTTR, availability	Static (native K8s healing)	User-space (observability)	Reactive	Case study/demo	General enterprise
[14]	OS container (K8s)	P99 latency, consistency	Rule-based, threshold+hysteresis	User-space middleware	N/A	Fault-injection + statistics	E-commerce
[15]	OS container (K8s, dual)	Throughput, p95 latency	Reactive, policy threshold	User-space middleware	Reactive	Bare-metal, dual-cluster	Enterprise hybrid-cloud
[16]	OS container + bare-metal edge	Task duration, std. dev.	Adaptive heuristic (EWMA)	User-space (above K8s)	Partition-level resize	Bare-metal, real failure	Video analytics/edge AI
[17]	OS container (Docker Swarm)	Voting rounds, failure rate	Real-time control loop + SVC	User-space middleware	Adaptive quorum	Emulated Swarm tasks, scripted faults	IoT/edge sensing
[47]	OS container (K8s, CRI-O)	Execution time, container restarts, request success	Static priority model	Node-level agent via CRI	Vertical memory scaling, suspend and checkpoint	Bare-metal, four-node cluster	Stateful batch and stateless services
[19]	Host/device level	Failure freq., downtime	Static, two-step heartbeat	User-space middleware	N/A	90-day field deployment	Clinical support
[23]	OS container (GKE)	RTO, RPO, throughput	Static (Raft consensus)	User-space middleware	Failure-triggered	Managed cloud (GKE), network emulation	Enterprise OLTP
[25]	OS container (Alibaba K8s)	RTO, throughput	Static, lease state machine	Kernel + user-space	Failure-triggered	Production cloud cluster	Cloud messaging
[26]	Hybrid: VM-TEE + container	Control/data-plane latency	Static, guard state machine	Control plane + kernel	N/A	Bare-metal + simulation	Multi-tenant cloud
[48]	OS container (service under test)	Vulnerabilities found, bug coverage	Rule-based test prioritisation and threshold oracles	User-space testing framework	N/A	Production deployment and 48-bug benchmark	Production cloud microservices
[27]	N/A (network emulation testbed)	Success rate, latency, elections	N/A (studies Raft/etcd)	System coordination substrate	N/A	Network emulation	General coordination
[29]	OS container (K8s)	MTTR, success rate, energy	Hybrid: anomaly detection + RL	User-space	Failure-triggered	Fault-injection benchmark	Web applications
[30]	OS container (K8s, Pi, GPU)	SLO violations, MTTR	Deep RL (PPO)	User-space middleware	N/A	Bare-metal, three environments	Mixed: web, edge, AI
[31]	OS container + OpenStack	Prediction accuracy, MTTR	Hybrid: supervised + threshold	User-space	Predictive	Log analysis + emulated cloud	General hybrid cloud

Table S2: Matrix 2: Empirical evaluation methodology and fault-injection parameters. Rows below the rule have a learned decision component.

Paper	Environment	Workload generator	Fault-injection tool	Fault parameters
[4]	Grid'5000, real physical cluster	ClusterLoader + Prometheus	Netem	Latency 0/50/250/400ms; loss up to 10%
[5]	Real GCP + AWS clusters	Custom workload generator + Kafka	Simulated packet loss/RTT	150–300ms RTT/1% loss; 800–1200ms/10% loss (severe)
[6]	Simulation (50 synthetic VNFs)	Custom synthetic VNF generator	None	Gradual resource-overload ramp; no network faults
[7]	4/8-node K8s, cloud+edge	Locust	Chaos Mesh	7 fault types, 4 intensities incl. network partition
[8]	2 heterogeneous testbeds (incl. Pi 4, Kria KV260)	Custom UDP echo, Open5GS+UERANSIM	K8s NoExecution taint	Taint-based exclusion; 5–20 pods/node, 1–3 dest. nodes
[9]	6× Raspberry Pi 4	Custom test harness	None	No fault injection; steady request rates only
[10]	3 heterogeneous edge nodes	Locust	None (load-triggered)	Dual-threshold CPU load only (τ_O=80%, τ_P=64%)
[11]	20-node heterogeneous GPU testbed	Trace replay	None (designed, untested)	Heartbeat 1s, 3-miss threshold; not experimentally triggered
[12]	Simulation, single dev machine	Own simulator	Simulated churn model	1–5% node failure rate; K8s/Hyperledger baselines modeled, not run
[13]	K8s (cluster details not given)	Own telemetry	LitmusChaos	Named, not quantified (pod term., CPU/mem., network)
[14]	K8s 1.29 + Kafka + PostgreSQL	k6	Chaos Mesh	5 fault types incl. partition; duration not quantified
[15]	Private + public K8s clusters	DeathStarBench, Train-Ticket	Not named	Broker outages, cross-cluster link failures
[16]	K8s + real Pi/Jetson devices	Real workloads (YOLOv8, transcoding)	Custom worker-disconnect injector	Real failure across 3 worker pairs, 20s reintegration
[17]	70 emulated Swarm tasks; hardware not reported	Custom query client	Custom probabilistic injector	1% message-drop; 0–32/70 nodes faulty
[47]	Four-node bare-metal K8s, 1 Gbps	Stateful benchmark applications; concurrent request workers	None	Memory pressure induced by workload: oversubscription up to 150%; normal, peak, and overload concurrency
[19]	2 real production servers	Live clinical traffic	Manual (by hand)	4 fault types, manual trigger
[23]	3-node GKE	pgbench	tc netem	Pod kill; 100% loss; kill 2/3 nodes in 5s
[25]	Real Alibaba Cloud K8s	Custom RocketMQ generator	ChaosBlade + manual port stop	Node crash, forced migration, I/O hang, port-stop "partition"
[26]	6-node AMD SEV-SNP cluster	K-Bench, app benchmarks	None (threat model only)	Arbitrary disconnection/clock manipulation, not fault-injected
[48]	Huawei Cloud production services; per-test isolated containers	Replayed production traffic	Application-level fault library	Software exceptions and inter-service communication errors; 137 potential vulnerabilities, 89 confirmed; 90% coverage of 48 reproduced bugs
[27]	Mininet, real etcd binary	Custom write-traffic generator	iptables	Leader crash, full/partial/intermittent partition
[29]	Infrastructure not reported	Custom Python monitoring service	Custom (unnamed)	6 fault types incl. network delay; ≥30 trials each
[30]	6-node K8s, 4-node Pi 5 edge, 4-node A100	Locust, wrk2	RL-driven fault selection	Latency 50–300ms, loss 0.1–5%, CPU 60–95%
[31]	Log analysis + emulated hybrid cloud	Not specified	Not specified	No fault parameters reported

reachability evidence, workload characteristics, and recovery safety before authorising recovery. This narrower statement is intentional because other papers in the corpus do implement secondary verification or safeguards in different contexts. *Matrix 3 – Causality Gap and Systems Evaluation Audit.*

The averages fallacy is the most frequently encountered trap, present or partial in 12 of the 24 papers; the localhost fallacy is comparatively rare, present cleanly in only [12], whose evaluation is a full simulation with no real network path at all. Of the two papers most central to the gap identified in this review, [23] avoids every applicable trap and [27] is partial on the averages fallacy only.

Matrix 4 – Sovereignty and Resource-Constrained Settings Fit.

[10] is the only paper scoring 4 or above on all four sub-dimensions, reflecting its explicit rural-care, intermittent-connectivity motivation. No paper scores above 3 on data residency except [10] and [26], the latter for reasons orthogonal to this project's concerns, namely protecting tenant data from an untrusted cloud provider via hardware trusted execution, rather than supporting offline clinical operation. This gap between high-hardware-efficiency papers ([16], scoring 5 on that sub-dimension) and high-residency papers is itself notable: the reviewed literature does not present a single mechanism strong on both constrained-hardware efficiency and data residency simultaneously, a combination directly relevant to the healthcare deployment context of this project.

Table S3: Matrix 3: Evaluation-quality appraisal. Y = fell into the trap (0); P = partial (0.5); N = avoided (1); N/A = not applicable and not scored. Score = total over applicable criteria; the reason is given for every Y or P cell. Rows below the first rule have a learned decision component; the last row applies the same criteria to this work.

Paper	Averages fallacy	Localhost fallacy	Infinite bandwidth	Score	Reason for Y or P
[4]	N	N	N	3/3	–
[5]	N	N	N	3/3	–
[6]	N	N/A	N/A	1/1	–
[7]	N	N	N	3/3	–
[8]	N	N	N/A	2/2	–
[9]	Y	N	N/A	1/2	Placement latency reported without variance.
[10]	N	N	P	2.5/3	Links between nodes neither throttled nor characterised.
[11]	P	N	N	2.5/3	Median and p99 for scheduling delay only; other metrics are single averages.
[12]	P	Y	N/A	0.5/2	p95 without repeat count or variance across seeds. All results simulated on one machine.
[13]	Y	N/A	N/A	0/1	Single values without repetition or spread.
[14]	N	N/A	N/A	1/1	–
[15]	P	N	N/A	1.5/2	p95 latency only; throughput, availability and loss are single values.
[16]	P	N	N	2.5/3	Recovery timings averaged without stated trial counts.
[17]	N	P	N/A	1.5/2	Message loss is a modelled probability, not a degraded path.
[47]	N	N	N	3/3	–
[19]	Y	N	N	2/3	Average detection times over five trials without spread.
[23]	N	N	N	3/3	–
[25]	Y	N	N/A	1/2	Single RTO values without stated trial counts.
[26]	Y	N	N/A	1/2	Single values per tenant count; no repeats or error bars.
[48]	N	N	N/A	2/2	–
[27]	P	N	N	2.5/3	Fault traces show one of five runs without variance.
[29]	P	N/A	N/A	0.5/1	Medians with 95% CI for two of six fault types only.
[30]	N	N	N	3/3	–
[31]	Y	N/A	N/A	0/1	Single point values; baselines not described.
This work	N	N	N	3/3	Per-run values, SD, 95% CI and range reported; separate virtual machines over real interfaces; degradation applied with `tc`.

Table S4: Matrix 4: Sovereignty and resource-constrained fit (1–5 scale; anchors in Table S5). Rows are ordered by the sum of the four sub-scores.

Paper	Offline autonomy	Bandwidth/egress	Data residency	Hardware efficiency
[10]	4	4	5	4
[12]	4	2	3	3
[16]	3	3	1	5
[19]	3	3	1	4
[4]	1	4	3	2
[5]	2	3	2	3
[7]	2	4	1	3
[17]	2	3	1	4
[27]	2	3	1	3
[8]	2	2	1	4
[15]	1	2	3	2
[30]	2	3	1	2
[26]	1	1	4	1
[9]	1	2	1	3
[6]	2	1	1	3
[23]	1	2	1	2
[47]	1	1	1	3
[25]	1	1	1	2
[29]	1	1	1	2
[48]	1	1	1	1
[11]	1	1	1	1
[13]	1	1	1	1
[14]	1	1	1	1
[31]	1	1	1	1

Table S5: Scoring anchors for Matrix 4. Scores of 2 and 4 lie between the neighbouring anchors.

Criterion	1	3	5
Offline autonomy	No offline or disconnected operation considered	Local operation designed for or tested under short disruptions	Sustained disconnected operation designed for and demonstrated
Bandwidth or egress	Not considered	Constrained or high-latency links part of the evaluation	Bandwidth or egress reduction a primary, measured objective
Data residency	Not discussed	Data locality or routing addressed indirectly	Local residency of sensitive data required by design
Hardware efficiency	Data-centre hardware; no constraint considered	Edge deployability profiled, or modest commodity hardware	Constrained devices used, with overhead measured

S2 Run-Level OpenMRS Tables

These tables give the run-level OpenMRS values summarised in Section IX of the main paper.

Table S6: OpenMRS Scenario B: original backend Pod identity in the 15 per-configuration runs.

Mode	Runs	Pod changed	Pod retained
V0	3	3	0
T5	3	3	0
T10	3	3	0
T30	3	3	0
FA1 (initial)	3	2	1

Table S7: OpenMRS Scenario B Failure-Aware runs: decision chain from verification to Pod outcome. Fault durations are from each run's recorded fault timestamps.

Run	Verification	Decision	Toleration	Fault (s)	Native lifecycle events	Original Pod
003	3/3 HTTP 200	PRESERVE	300 s	121.2	Taint-manager deletion cancelled	Retained
004	3/3 HTTP 200	PRESERVE	300 s	902.4	Taint-manager deletion; ReplicaSet created replacement on Worker2	Replaced
005	3/3 HTTP 200	PRESERVE	300 s	468.7	Taint-manager deletion; ReplicaSet created replacement on Worker2	Replaced
006 (validation)	3/3 HTTP 200	PRESERVE	Indefinite	468.0	None disruptive; Pod Running and Ready on Worker1 after the run	Retained

Table S8: OpenMRS component resource audit; usage is one observed snapshot.

Component	Observed CPU (m)	Observed memory (MiB)	CPU request (m)	Memory request (MiB)
Backend	19	929	750	1400
MariaDB	9	130	300	512
Frontend	1	2	100	96
Gateway	1	3	100	96
Total	30	1064	1250	2104

Table S9: Hypothetical additional OpenMRS backend memory reservation by worker (request-based snapshot, not observed concurrent duplication).

Worker	Allocatable (MiB)	Existing requests (MiB)	With backend (MiB)	Fits
Worker1	3915	2992	4392	No
Worker2	3915	1024	2424	Yes
Worker3	1968	0	1400	Yes
Worker4	1968	704	2104	No

Table S10: OpenMRS Scenario B clinical probe failures in the nine runs with samples. The interval runs from the first failed probe to the next successful one; "Open" means no successful probe followed, so the interval is censored.

Run	Failed probes	Interval (s)	Original Pod
V0/003	9	11.847	Replaced
V0/004	14	Open	Replaced
T5/003	6	Open	Replaced
T5/004	17	Open	Replaced
T10/003	6	17.795	Replaced
T10/004	24	Open	Replaced
T30/003	14	16.965	Replaced
FA1/003	1 of 102	1.209	Retained
FA1/004	19 of 677	23.269	Replaced

Table S11: OpenMRS Scenario C: fault duration, time from fault to first clinical restoration, and time from fault clearance to restoration, in seconds. Descriptive only.

Run	Fault duration	Fault to restoration	Clearance to restoration
V0/003	902.410	922.216	19.807
V0/004	908.435	927.227	18.792
T5/003	910.445	935.102	24.657
T5/004	904.425	931.693	27.268
T10/003	132.502	592.121	459.618[a]
T10/004	903.221	919.773	16.551
T30/003	908.235	931.060	22.825
T30/004	905.172	927.932	22.761
FA1/003	609.550	635.936	26.386
FA1/004	903.688	922.523	18.835
Configuration means ($n = 2$)			
V0	905.423	924.722	19.300
T5	907.435	933.398	25.963
T10	517.862	755.947	238.085
T30	906.704	929.496	22.793
FA1	756.619	779.230	22.611

[a] Cause of the interval not verified.

Table S12: OpenMRS Scenario C+D intervals (s), primary cohort $n = 9$ (runs 011 and 013–020).

Interval	Mean ± SD	Median	Range
Fault → database probe unreachable[†]	3.98 ± 0.79	3.53	3.45–5.83
Fault → RECOVER decision	57.57 ± 2.29	57.11	54.30–61.45
Decision → fence verified	1.80 ± 0.24	1.74	1.63–2.43
Fence verified → recovery verified	102.10 ± 8.74	106.23	89.41–111.37
Fault → recovery verified	161.48 ± 8.95	165.16	149.53–170.95
Reconciliation	575.56 ± 26.60	584.00	505–588

[†] The independent database probe on the verification path, which loses the database within seconds of the fault. Native suspicion (`Ready=Unknown`) follows about 50 s later, and only then does the controller verify.

Table S13: Experiment inventory for both evaluations: attempted, aborted and analysed runs. Second-phase runs are listed in Section S3.

Workload	Campaign	Configurations	Runs	Note
Nginx	Scenario B	Native, FA1	30, 32	Control-plane partition; workload reachable. FA1 run 019 aborted after 3.8 s, before suspicion.
Nginx	Scenario B demonstration	FA1	4	Runs 201, 301, 302, 501: all retained with PRESERVE; not analysed.
Nginx	Scenario C	Native, FA1	30, 30	Control-plane and client paths isolated.
Nginx	Scenario A	Native, FA1	5, 5	Worker1 shut down; 6 and 7 attempts, incomplete attempts excluded.
Nginx	Single-node baseline	No orchestration	10	Host Nginx on Worker1, observed in every Scenario A run.
Nginx	Scenario D (PostgreSQL)	FA1	31	Old writer stopped before fencing in every run.
Nginx	Scenario E	FA1: mild, moderate, severe	10, 10, 3	Control-plane path degraded on the worker's interface.
Nginx	Overhead	Native, FA1	30, 30	Normal operation; first-phase campaign.
Nginx	Candidate selection	FA1	4	Controlled cases with configured reliability scores.
OpenMRS	Scenario B	V0, T5, T10, T30, FA1	15	Three per configuration.
OpenMRS	Scenario B run 006	FA1	1	Validation with non-expiring tolerations, reported separately.
OpenMRS	Scenario C	V0, T5, T10, T30, FA1	10	Two per configuration.
OpenMRS	Scenario E	Five configurations, three profiles	14	Severe observations for V0, T5, T30, and FA1.
OpenMRS	Scenario C+D runs 011–020	FA1	10	MariaDB on NFS with storage fence and SSH reconciliation; run 012 interrupted after recovery.
OpenMRS	Scenario C+D run 010	FA1	0	Stopped at preflight (Worker3 cordoned); no fault applied.
OpenMRS	Scenario C+D pilots	FA1	4	Fencing and recovery verified in all four; not pooled.
OpenMRS	Scenario C+D run 003	FA1	1	Trial run before the campaign, without the reconciliation stage; the campaign runs are numbered from 010.

S3 Experiment Inventory and Second-Phase Campaigns

S4 Original-Campaign Result Tables

The tables in this section are referred to from the main text by number.

References

[1] The Kubernetes Authors, "Nodes," Kubernetes Documentation. [Online]. Available: `https://kubernetes.io/docs/concepts/architecture/nodes/` (accessed Oct. 28, 2026).

[2] The Kubernetes Authors, "Taints and Tolerations," Kubernetes Documentation. [Online]. Available: `https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/` (accessed Oct. 11, 2026).

[3] The Kubernetes Authors, "Kubernetes 1.28: Non-Graceful Node Shutdown Moves to GA," Kubernetes Blog, Aug. 16, 2023. [Online]. Available: `https://kubernetes.io/blog/2023/08/16/kubernetes-1-28-non-graceful-node-shutdown-ga/` (accessed Oct. 11, 2026).

[4] K. Manaouil and A. Lebre, "Kubernetes WANWide: a Deployment Scenario to Expose and Use Edge Computing Resources," in Proc. 2021 29th Euromicro Int. Conf. Parallel Distrib. Netw.-Based Process. (PDP), 2021, pp. 193-197, doi: 10.1109/PDP52278.2021.00038.

[5] P. Kedar and V. Mulpuri, "Building An Elastic, Resilient, Scalable Cloud-Native Digital Twin by Leveraging Stream Adaptation Agents for Smart Distributed Manufacturing Systems," in Proc. 2026 IEEE Cloud Summit, 2026, pp. 64-69, doi: 10.1109/CloudSummit68932.2026.00017.

[6] B. Tola, "Self-Healing for Resilient NFV Orchestration: A Predict-Policy-Act Approach," in Proc. 2026 Int. Conf. Smart Appl. Commun. Netw. (SmartNets), 2026, doi: 10.1109/SMARTNETS69662.2026.11604864.

Table S14: Second-phase campaigns (commit `4ce8078`, extension addendum `ddfc677`).

Campaign	Arms and conditions	Runs	Status
Baseline ladder (seed 3106524066)	V0, T10, NX, R2, FA1; reachable 120 and 600 s, unreachable 480 s	75	75 completed, 0 aborted
Ladder, second set (seed 781441089)	V0, T10, NX, R2, FA1; reachable 600 s, unreachable 480 s	30	30 completed, 0 aborted; pooled with the baseline ladder
Ladder, third set (seed 472857209)	V0, T10, NX, R2, FA1; reachable 120 s	15	15 completed, 0 aborted; pooled with the baseline ladder
Probe grid	Six workload states, 100 trials each, $k = 1, 3, 5$	600 trials	Completed
Overhead	30 idle windows with and 30 without the controller	60 windows	Completed
Live writer, first campaign (seed 3063071617)	Planned 10 fenced and 10 unfenced	20	20 completed; no fence applied in any run; reported as unfenced
Live writer, fenced (seed 721711068)	Fenced	10	10 completed, 0 aborted
Live writer, fenced (seed 2559281726)	Fenced	10	10 completed, 0 aborted
Probe grid, extended (seed 781441089)	Reply delay 0.5, 1.0, 1.5 s; reply loss 70 and 90%; 100 trials each, $k = 1, 3, 5$	500 trials	Completed
Extension E1 (seed 2000988837)	T5, T30; reachable 120 and 600 s	20	20 completed, 0 aborted
Extension E2 (seed 3566506833)	V0, T10, NX, FA1; reachable 15, 60, 300 s	36	36 completed, 0 aborted
OpenMRS Scenario B under load (runs 101–103)	V0, T5, T10, T30, FA1; 600-s partition; ten virtual clinical users	14	14 completed, 0 aborted (T30: two runs)

Table S15: Scenario B aggregate results. Medians carry BCa bootstrap 95% intervals (10,000 resamples); p95 and p99 are given where $n \geq 20$; with $n \approx 30$, p99 lies near the maximum. A dash denotes an inapplicable or unavailable value. FA1 run 019 (aborted before suspicion) is excluded. [§]Run 003 is excluded: no observation was recorded for 625.9 s, so its outage is unobserved.

Mode	Metric	n	Mean (s)	SD (s)	Median [BCa 95% CI] (s)	p95 (s)	p99 (s)	Range (s)
Native	Replacement pending	24	347.83	3.28	347.24 [345.64, 349.31]	353.28	353.56	[341.66, 353.58]
Native	Replacement running	30	349.87	3.62	349.76 [347.77, 352.02]	355.03	355.79	[342.83, 355.91]
Native	HTTP outage	8	4.57	0.37	4.42 [4.29, 4.87]	–	–	[4.22, 5.34]
Failure-Aware	Suspicion delay	32	51.36	18.72	47.52 [46.94, 49.33]	54.45	122.18	[41.76, 152.29]
Failure-Aware	Decision delay	32	20.94	110.37	1.35 [1.20, 1.42]	2.23	432.52	[1.12, 625.79]
Failure-Aware	HTTP outage[§]	4	115.71	90.23	114.55 [6.57, 227.19]	–	–	[6.57, 227.19]
Failure-Aware	Replacement/service recovery	0	–	–	–	–	–	–

[7] Z. Chen, M. Goudarzi, and A. N. Toosi, "An Empirical Evaluation of Kubernetes Resilience in Cloud-Edge Deployments Using Failure Injection Techniques," in Proc. 2026 Australasian Comput. Sci. Week (ACSW), 2026, doi: 10.1145/3793811.3793823.

[8] M. Barletta, M. Cinque, and L. De Simone, "SLO-aware prioritization of orchestration times for containerized services," ACM Trans. Internet Technol., vol. 26, no. 1, Art. no. 13, pp. 1-29, Jan. 2026, doi: 10.1145/3767329.

[9] G. Morabito, A. Ficara, A. Celesti, M. Villari, and M. Fazio, "Consensus-based distributed orchestration framework for microservices in edge computing clusters," Future Gener. Comput. Syst., vol. 176, Art. no. 108221, 2026, doi: 10.1016/j.future.2025.108221.

[10] P. Rodriguez, J. Mateos-Bravo, S. Laso, J. L. Herrera, and J. Berrocal, "Predictively controlling the computing continuum with distributed energy-aware orchestration," J. Syst. Archit., vol. 175, Art. no. 103752, 2026, doi: 10.1016/j.sysarc.2026.103752.

[11] D. Gao, G. Dai, L. Tian, Y. Wang, L. Guo, and C. Zheng, "DSFRO: Dynamic scheduling and fine-grained resource optimization for distributed large model training in heterogeneous clusters," IEEE Access, vol. 14, pp. 59409-59421, 2026, doi: 10.1109/ACCESS.2026.3681290.

[12] C. Keum, Y. Song, S. Lee, K. Cho, and H. Bahn, "Mutual Cloud: Decentralized Task Orchestration in Loosely Coupled Distributed Environments," Appl. Sci., vol. 16, no. 7, Art. no. 3484, 2026, doi: 10.3390/app16073484.

Table S16: Scenario C aggregate results. Medians carry BCa bootstrap 95% intervals (10,000 resamples); p95 and p99 are given where $n \geq 20$; with $n = 30$, p99 lies near the maximum.

Mode	Metric	n	Mean (s)	SD (s)	Median [BCa 95% CI] (s)	p95 (s)	p99 (s)	Range (s)
Native	Replacement pending	25	349.14	3.69	349.43 [346.54, 350.54]	355.71	355.95	[342.98, 356.03]
Native	Replacement running	30	353.97	18.49	350.65 [347.89, 352.38]	358.26	423.25	[344.17, 449.77]
Native	End-to-end recovery (ERL)	30	356.56	18.36	352.84 [350.80, 355.63]	361.80	425.47	[346.11, 451.22]
Native	HTTP outage (SID)	30	374.66	116.70	353.10 [351.31, 355.44]	361.64	809.62	[347.24, 992.13]
Failure-Aware	Suspicion delay (FDL)	30	50.15	3.78	51.47 [48.58, 52.19]	55.10	55.61	[43.05, 55.70]
Failure-Aware	Decision delay	30	10.81	1.06	10.26 [10.13, 10.34]	12.85	13.18	[10.08, 13.28]
Failure-Aware	Recovery-release delay	30	0.08	0.16	0.03 [0.02, 0.06]	0.21	0.69	[0.01, 0.86]
Failure-Aware	Replacement pending	18	61.80	3.31	62.51 [59.51, 63.59]	–	–	[55.05, 67.01]
Failure-Aware	Replacement running	30	63.03	3.46	64.12 [62.35, 64.74]	67.31	68.02	[55.03, 68.31]
Failure-Aware	End-to-end recovery (ERL)	30	65.43	3.22	66.11 [65.12, 66.63]	69.22	71.92	[57.66, 73.01]
Failure-Aware	HTTP outage (SID)	30	62.69	3.66	62.99 [60.88, 64.08]	68.38	68.78	[55.50, 68.90]

Table S17: Scenario D storage-authority correctness and timing results. Intervals are Student-t 95% intervals for the mean.

Metric	n	Mean (s)	SD (s)	95% CI (s)	Range (s)
Old-authority stopping time	31	3.38	0.78	[3.09, 3.67]	[2.05, 4.69]
Fence-to-recovery-release time	31	11.88	9.79	[8.29, 15.47]	[9.09, 64.36]
Recovery-release-to-service time	31	3.93	1.22	[3.48, 4.38]	[2.46, 9.03]
Fence-to-service time	31	15.82	10.82	[11.85, 19.79]	[11.67, 73.39]

[13] G. Gurudu and A. Devineni, "Chaos Engineering and Observability Integration for Resilient Cloud Infrastructure," in Proc. 2026 Int. Conf. Intell. Sustainable AI Syst. (ICOSAAS), 2026, pp. 1137-1144, doi: 10.1109/ICOSAAS68663.2026.11648619.

[14] J. Chen, "Distributed transaction processing and consistency guarantee of e-commerce transaction systems under cloud-native architecture," in Proc. 2026 5th Int. Conf. Distrib. Comput. Electr. Circuits Electron. (ICDCECE), 2026, doi: 10.1109/ICDCECE68961.2026.11645832.

[15] S. R. Bongu, "Scalable Middleware Layer for Secure, Fault-Tolerant, and High-Throughput Enterprise Application Integration Across Hybrid Cloud Environments," in Proc. 2026 1st Int. Conf. Artif. Intell. Mach. Learn. Commun. Power Syst. (AIMLCPS), 2026, doi: 10.1109/AIMLCPS68702.2026.11542477.

[16] S. Kwon, Y. Noh, Y. Jang, I. Choi, B. Tak, and Y.-K. Suh, "Scalable and robust resource provisioning via adaptive task scheduling for edge devices," IEEE Trans. Netw. Serv. Manag., early access, 2026, doi: 10.1109/TNSM.2026.3694238.

[17] G. Javens and K. Ravindran, "Adaptive Threshold Control for Replicated Edge Services Under Uncertain Faults," in Proc. 2026 2nd Int. Conf. Emerging Intell. Technol. Syst. (EITS), 2026, pp. 288-294, doi: 10.1109/EITS70066.2026.11609628.

[18] A. Gomez-Gonzalez, C. Carrion, and M. B. Caminero, "Enhancing fog IoT container deployment: A customizable Kubernetes scheduler," Future Gener. Comput. Syst., vol. 176, Art. no. 108195, 2026, doi: 10.1016/j.future.2025.108195.

[19] M. H. Naim, J. M. Zain, M. K. Abd Ghani, K. Abd Jalil, and M. H. Jamari, "Double check method: An enhancement of heartbeat failure detection by fog devices through socket and port engagement," J. Netw. Comput. Appl., vol. 253, Art. no. 104538, 2026, doi: 10.1016/j.jnca.2026.104538.

[20] H. Boudlal, M. Serrhini, and A. Tahiri, "Cloud computing for healthcare services: Technology, application and architecture," in Proc. 11th Int. Symp. Signal, Image, Video Commun. (ISIVC), 2022, doi: 10.1109/ISIVC54825.2022.9800212.

Table S18: Controlled multi-candidate recovery-selection checks. Each case is a single controlled run on the live cluster with supplied reliability inputs.

Test	Controlled condition	Selected candidate	Observed outcome
Reliability selection	All candidates Ready, schedulable, and resource-feasible	`k3s-worker3`	Highest reliability score (0.997000) selected despite `k3s-worker2` exposing twice its allocatable CPU.
Resource exclusion	`k3s-worker3` requested 1950m of 2000m CPU, leaving 50m for a 100m recovery requirement	`k3s-worker4`	Worker3 was correctly marked ineligible despite having the highest reliability score.
Availability exclusion	`k3s-worker3` remained Ready but was cordoned (`schedulable=false`)	`k3s-worker4`	The highest-reliability node was excluded and the next-best eligible candidate was selected.
RECOVER placement	All candidates eligible; original workload classified `Unreachable` after three failed probes	`k3s-worker3`	Controller emitted RECOVER, constrained the Deployment to Worker3, released the Worker1 Pod, and the replacement became Running on Worker3.

Table S19: Scenario E outcomes under degraded control-plane networking.

Profile	Runs	`Ready=Unknown`	PRESERVE	Replacement	Original Pod retained	Decision scored
Mild	10	0/10	Not triggered	0/10	10/10	Not scored (non-event)
Moderate	10	0/10	Not triggered	0/10	10/10	Not scored (non-event)
Severe	3	3/3	3/3	0/3	3/3	3/3 correct

[21] A. Priyadarshini, S. K. Pradhan, S. R. Laha, and D. S. K. Nayak, "Enhancing fault tolerance and load balancing in cloud computing for improved e-healthcare systems performance," in Proc. 2nd Int. Conf. Ambient Intell. Health Care (ICAIHC), 2023, doi: 10.1109/ICAIHC59020.2023.10431436.

[22] A. Srivastava, A. Pal, and V. Ilango, "Security issues in cloud computing for healthcare," in Proc. IEEE Int. Conf. Intell. Syst., Smart Green Technol. (ICISSGT), 2026, doi: 10.1109/ICISSGT67741.2026.11483746.

[23] I. Jerrari and I. Assayad, "Comparative Analysis of High Availability Database Systems in Containerized Environments: Performance, Resilience, and Recovery Time Optimization," in Proc. 2026 6th Int. Conf. Innovative Res. Appl. Sci. Eng. Technol. (IRASET), 2026, doi: 10.1109/IRASET68627.2026.11538890.

[24] J. Redzepagic, A. Kapulica, N. Malesevic, and V. Dakic, "Empirical performance and operational analysis of monolithic and distributed database architectures in Kubernetes environments," Computers, vol. 15, no. 5, Art. no. 282, 2026, doi: 10.3390/computers15050282.

[25] R. Jin, Y. Gu, J. Ji, Y. Fu, T. Liu, F. Lai, G. Cai, and Q. Lin, "Replication-Free Failover: Protocol-Fenced Takeover for Stateful Services," in Proc. 34th ACM Int. Conf. Foundations Softw. Eng. (FSE Companion), Montreal, QC, Canada, 2026, pp. 536-544, doi: 10.1145/3803437.3805226.

[26] X. Li, W. Liu, F. Li, H. Tian, Z. Liu, S. Yan, and M. Gao, "Pyramid: A Secure, Resource-Efficient, and Pluggable Kubernetes for Multi-Tenancy," in Proc. 21st Eur. Conf. Comput. Syst. (EuroSys), Edinburgh, Scotland, UK, 2026, pp. 311-327, doi: 10.1145/3767295.3769324.

[27] C. Jensen, H. Howard, and R. Mortier, "Examining Raft's behaviour during partial network failures," in Proc. 1st Workshop High Availability Observability Cloud Syst. (HAOC), Apr. 2021, 7 pages, doi: 10.1145/3447851.3458739.

[28] S. Alam, A. Ullah, and Z. Wang, "NL-CPS: Reinforcement Learning-Based Kubernetes Control Plane Placement in Multi-Region Clusters," in Proc. 2026 IEEE 10th Int. Conf. Fog Edge Comput. (ICFEC), 2026, pp. 57-64, doi: 10.1109/ICFEC69006.2026.00015.

Table S20: OpenMRS Scenario B fault duration of each run, from the recorded fault-start and fault-clear timestamps, against the approximate native eviction window.

Mode	Window (s)	Run 003	Run 004	Run 005	Retained
V0	≈350	902.5	422.3	460.1	0/3
T5	≈55	122.3	422.4	468.4	0/3
T10	≈60	602.8	422.7	459.8	0/3
T30	≈80	902.3	460.1	467.5	0/3
FA1 (initial)	≈350	121.2	902.4	468.7	1/3 (run 003)

[29] M. L. Dsouza, S. N. Kumaraswamy, and S. L. Dsouza, "A study on emerging artificial intelligence for sustainable self-healing web applications in the cloud era," in Proc. 2026 Int. Conf. Emerging Trends Innov. ICT (ICEI), Pune, India, 2026, pp. 1-4, doi: 10.1109/ICEI65890.2026.11448042.

[30] S. Nigam, S. Patel, and A. I. Khan, "AI-Augmented Chaos Testing: Toward Intelligent Resilience Engineering in Cloud-Native Systems," in Proc. 2026 Int. Conf. Sustain. Eng. Digit. Innov. (ICSEDI), 2026, doi: 10.1109/ICSEDI66420.2026.11568339.

[31] N. Mishra and A. Soy, "AI-Driven Fault Tolerance Mechanisms for Real-Time Cloud Service Recovery in Hybrid Environments," in Proc. 2026 6th Int. Conf. Expert Clouds Appl. (ICOECA), 2026, pp. 1188-1193, doi: 10.1109/ICOECA68095.2026.11485160.

[32] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," J. ACM, vol. 43, no. 2, pp. 225–267, 1996, doi: 10.1145/226643.226647.

[33] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, "The ϕ accrual failure detector," in Proc. 23rd IEEE Int. Symp. Reliable Distrib. Syst. (SRDS), 2004, pp. 66–78, doi: 10.1109/RELDIS.2004.1353004.

[34] A. Das, I. Gupta, and A. Motivala, "SWIM: Scalable weakly-consistent infection-style process group membership protocol," in Proc. Int. Conf. Dependable Syst. Netw. (DSN), 2002, pp. 303–312, doi: 10.1109/DSN.2002.1028914.

[35] A. Dadgar, J. Phillips, and J. Currey, "Lifeguard: Local health awareness for more accurate failure detection," arXiv:1707.00788, 2017, doi: 10.48550/arXiv.1707.00788.

[36] P. Huang, C. Guo, L. Zhou, J. R. Lorch, Y. Dang, M. Chintalapati, and R. Yao, "Gray failure: The Achilles' heel of cloud-scale systems," in Proc. 16th Workshop Hot Topics Oper. Syst. (HotOS), 2017, pp. 150–155, doi: 10.1145/3102980.3103005.

[37] A. Alquraan, H. Takruri, M. Alfatafta, and S. Al-Kiswany, "An analysis of network-partitioning failures in cloud systems," in Proc. 13th USENIX Symp. Oper. Syst. Des. Implement. (OSDI), Carlsbad, CA, USA, 2018, pp. 51–68. [Online]. Available: `https://www.usenix.org/conference/osdi18/presentation/alquraan`

[38] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in Proc. USENIX Annu. Tech. Conf. (ATC), Philadelphia, PA, USA, 2014, pp. 305–319. [Online]. Available: `https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro`

[39] M. Kleppmann, "How to do distributed locking," Feb. 2016. [Online]. Available: `https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html` (accessed Oct. 28, 2026).

[40] Medik8s Project, "Self Node Remediation Operator," GitHub repository. [Online]. Available: `https://github.com/medik8s/self-node-remediation` (accessed Oct. 28, 2026).

[41] Patroni Contributors, "Patroni documentation." [Online]. Available: `https://patroni.readthedocs.io/` (accessed Oct. 28, 2026).

[42] The Kubernetes Authors, "kube-controller-manager," Kubernetes Documentation, command-line tools reference. [Online]. Available: `https://kubernetes.io/docs/reference/command-line-tools-reference/kube-controller-manager/` (accessed Oct. 15, 2026).

[43] The Kubernetes Authors, "Disruptions," Kubernetes Documentation. [Online]. Available: `https://kubernetes.io/docs/concepts/workloads/pods/disruptions/` (accessed Oct. 15, 2026).

[44] Medik8s Project, "Node Healthcheck Operator," GitHub repository. [Online]. Available: https://github.com/medik8s/node-healthcheck-operator (accessed Oct. 15, 2026).

[45] Red Hat, "Fencing," in *Red Hat Enterprise Linux 7 High Availability Add-On Overview*, Red Hat Documentation. [Online]. Available: https://docs.redhat.com/en/documentation/Red_Hat_Enterprise_Linux/7/html/high_availability_add-on_overview/s1-fencing-haao (accessed Oct. 28, 2026).

[46] E. Bainomugisha and A. Mwotil, "Crane Cloud: A resilient multi-cloud service layer for resource constrained settings," Makerere University, Kampala, Uganda, 2021. [Online]. Available: https://nru.uncst.go.ug/handle/123456789/7268 (accessed Oct. 28, 2026).

[47] T. Kang, M. Kang, and H. Yu, "Vertical auto-scaling mechanism for elastic memory management of containerized applications in Kubernetes," Future Gener. Comput. Syst., vol. 180, Art. no. 108407, 2026, doi: 10.1016/j.future.2026.108407.

[48] Z. Chen, Z. Deng, K. Zhang, Y. Liu, C. Cui, J. Zhong, and Z. Zheng, "CAST: Automated resilience testing for production cloud service systems," in Proc. IEEE/ACM 48th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP), 2026, doi: 10.1145/3786583.3786867.

[49] S. Mohanadas and L. P. Gorrepati, "COSMOS-R^2: Artificial intelligence–driven risk-bounded carbon-constrained autonomous cloud scaling via CMDP control," in Proc. 2026 IEEE Cloud Summit, 2026, doi: 10.1109/CloudSummit68932.2026.00015.

Table S21: Principal findings with exact (Clopper–Pearson) 95% intervals. "Window" is the native grace period plus the Pod's toleration.

	Finding	Evidence	Strength
F1	Native Kubernetes replaces a reachable workload once a partition outlasts the window.	Nginx: 30/30 replaced (retention 0%, 95% CI 0–11.6%). OpenMRS: 12/12 native and tuned runs replaced (0–26.5%); 14/14 runs longer than their window replaced.	Strong; repeated on two workloads.
F2	The controller classifies a reachable workload correctly and chooses PRESERVE.	Nginx: PRESERVE in 32 of 32 valid runs (95% CI 89.1–100%), no RECOVER; one further attempt (run 019) was aborted before suspicion could occur. OpenMRS: 12/12 verification attempts succeeded; PRESERVE in 4/4 runs (95% CI 39.8–100%).	Strong for Nginx; consistent but small for OpenMRS.
F3	The integrated configuration retains the workload.	Nginx: 32/32 retained (89.1–100%) against 0/30 natively, a difference of 100 points (84.4–100), Fisher exact $p = 2.2 \times 10^{-18}$. OpenMRS run 006 repeats the Nginx outcome on the healthcare workload.	Strong; established on Nginx and replicated on OpenMRS.
F4	A PRESERVE decision does not by itself stop native eviction.	OpenMRS: both FA1 runs that outlasted a 300-second window were evicted despite PRESERVE.	Clear mechanism; two runs.
F5	Avoiding a duplicate backend avoids a substantial reservation.	OpenMRS backend: 929 MiB used, 1,400 MiB and 750m CPU requested. Every replacement (12 native and tuned runs, FA1 runs 004 and 005) was placed on Worker2, taking 48% of its unreserved memory; Worker4 could not admit one.	Request-based; client cost under load in Scenario B runs 101–103.
F6	The controller releases an unreachable workload for recovery on an eligible candidate.	Nginx: RECOVER and a running replacement in 30/30 runs. OpenMRS: RECOVER in 2/2 runs, replacement placed on the selected worker, clinical service restored.	Strong for Nginx; consistent for OpenMRS.
F7	Storage is fenced before a duplication-sensitive workload is released and stays fenced until the original writer is retired.	PostgreSQL: ordering held in 31/31 runs (88.8–100%) with the writer stopped first. MariaDB: fence verified before replacement and released only after the original process was verified absent in 10/10 runs (69.2–100%), with the original runtime not stopped first.	Ordering held in every run.
F8	Removal of a Pod from Kubernetes does not mean its process has exited.	MariaDB run 003: original process in state `D` for more than 14 hours after its Pod disappeared from the API; runs 011–020 retired it by SSH reboot before releasing the fence (mean reconciliation 575.56 s).	Observed; reconciliation repeatable in 10 runs.
F9	Under genuine worker shutdown the controller classifies the workload correctly and, when a verified destination exists, restores service sooner than native K3s or a single node.	Nginx Scenario A: unreachable in 5/5; RECOVER with replacement on the selected node in 4/5 (28.4–99.5%), median outage 61.36 s over those four, against 350.06 s natively and 530.88 s for a single node. In run 004 no verified destination was available, the workload was held, and the outage was 592.37 s; the five-run median is 62.43 s.	Five runs per arm; wide intervals.